Resumo Aprofundado: Sistemas Distribuídos

Unidade 1: Introdução a Sistemas Distribuídos

Esta unidade estabelece a fundação dos sistemas distribuídos, abordando sua definição, componentes, modelos arquiteturais e o papel central da comunicação em rede[111111111].

1. Conceitos Fundamentais de Sistemas Distribuídos

- **Definição:** Um sistema distribuído é composto por **múltiplos computadores** (nós) que se c□municam e c□□rdenam para atingir um □bjetiv□ c□mum, □perand□ c□m□ uma **única unidade funcional**[2].
- **Vantagens (Principais):** Oferecem **maior disponibilidade** e **confiabilidade** (a falha de um componente não para o sistema) [3], e alta

escalabilidade (novas máquinas podem ser adicionadas para atender à demanda)[4].

- **Desvantagens/Desafios:** Incluem **complexidade de implementação** e **segurança** (vulnerabilidade a ameaças de rede)[5555]. A

sincronização entre processos é crítica e desafiadora, dada a **ausência de um relógio global**[6].

- **Aplicações Práticas:** São essenciais em diversos setores, como o **financeiro** (alta disponibilidade para transações 24/7) [7],

e-commerce (resiliência a picos de acesso) [8]e

streaming/mídia (disponibilidade global de conteúdo)[9].

2. Modelos de Sistemas Distribuídos

A arquitetura de um sistema distribuído define como seus componentes se relacionam[10101010].

- **Modelo Cliente-Servidor:** Caracteriza-se por ter uma **máquina central (servidor)** que fornece serviços e dados para as demais máquinas (clientes)[11111111]. Oferece

controle centralizado e **gestão estruturada** [12], mas o servidor central pode se tornar um ponto de falha[13].

- **Modelo Peer-to-Peer (P2P):** Todos os **nós (máquinas)** possuem **direitos e responsabilidades equivalentes** (atuam como clientes e servidores)[14141414]. Isso promove

alta escalabilidade, **descentralização** e **resiliência** [15151515], mas aumenta a complexidade de gestão[16].

- **Modelos de Comunicação:**
    - **Comunicação Síncrona:** O processo emissor aguarda a resposta do receptor antes de continuar[1717]. É ideal para

transações financeiras onde a **ordem e a consistência imediata** são críticas[18181818].

- **Comunicação Assíncrona:** Os processos operam de forma independente, sem esperar uma resposta imediata[19]. É mais rápida e flexível, adequada para sistemas que priorizam

escalabilidade e **flexibilidade**[20].

- **Técnicas de Resiliência:**
  - **Replicação de Dados:** Cria cópias de dados em múltiplas máquinas para garantir resiliência[21].
  - **Checkpointing:** Estratégia em que o **estado atual do sistema é salvo** em intervalos regulares, permitindo que o sistema seja restaurado ao último ponto de verificação, minimizando a perda de trabalho[22222222].

3. Redes de Comunicação em Sistemas Distribuídos

- **Protocolo TCP/IP:** É o pilar da comunicação[23]. O

TCP garante a **confiabilidade** e a **entrega ordenada** dos dados [24242424], sendo ideal para transações que exigem precisão[25]. O

IP é responsável pelo **endereçamento** dos pacotes[26].

- **Protocolo UDP:** Oferece comunicação mais **rápida**, mas **sem garantias de entrega**[27]. É usado em cenários onde a velocidade é mais importante, como em

streaming de vídeo e jogos online[28282828].

- **RPC (Remote Procedure Call):** Permite que um processo em uma máquina **execute uma função em outra máquina como se fosse local**[29292929]. Simplifica a comunicação entre componentes distribuídos[30].
- **Latência e Mitigação:** A latência (atraso na transmissão) é um desafio comum[31]. É mitigada por:
  - **Cache de Dados:** Armazena dados próximos ao usuário para acesso mais rápido[32].
  - **Balanceamento de Carga:** Distribui o tráfego entre múltiplos servidores para evitar sobrecarga[33].
  - **Criptografia (SSL/TLS):** Protocolos de segurança que protegem a transmissão de dados confidenciais[34].

4. Processos e Threads em Sistemas Distribuídos

- **Processos:** Unidade de execução **independente** com seus próprios recursos e memória[35]. Em sistemas distribuídos, dividem tarefas entre diferentes máquinas[36].
- **Threads: "Subprocessos"** dentro de um processo que **compartilham a mesma memória e recursos**[37373737]. Permitem a execução de múltiplas tarefas simultaneamente dentro do mesmo processo[38].

- **Comunicação Interprocessos (IPC):** Mecanismos que permitem a troca de informações e coordenação entre processos (ex.: troca de mensagens, buffers compartilhados ou RPC)[39].
- **Controle de Concorrência:** Essencial para evitar que múltiplos processos/threads acessem os mesmos recursos simultaneamente, o que poderia causar inconsistências[40]. Mecanismos comuns incluem

semáforos e **mutexes** (bloqueios exclusivos)[41].

Unidade 2: Coordenação e Sincronização

Esta unidade aprofunda os mecanismos que garantem a ordem e a integridade das operações em sistemas distribuídos, onde a coordenação descentralizada é um desafio[42].

1. Coordenação de Processos

- **Exclusão Mútua:** Técnica fundamental que garante que **apenas um processo acesse um recurso crítico por vez**, evitando conflitos e inconsistências[43434343]. Em sistemas distribuídos, exige soluções que funcionem de forma descentralizada[44].
- **Problemas Clássicos de Sincronização:**
    - **Produtor-Consumidor:** Produtor gera itens e o consumidor os processa[45].
    - **Leitores-Escritores:** Leitores acessam informações, enquanto escritores as modificam; escritores precisam de **acesso exclusivo** para evitar dados inconsistentes[46464646].
- **Mecanismos de Sincronização:**
    - **Semáforos:** Atuam como sinalizadores, controlando o acesso a recursos compartilhados. Os

binários permitem acesso exclusivo e os **contadores** limitam acessos simultâneos[47474747].

    - **Monitores:** Encapsulam métodos e dados compartilhados, oferecendo **exclusão mútua automática** e simplificando a programação concorrente[48484848].

2. Algoritmos de Sincronização Distribuída

Esses algoritmos resolvem o desafio de ordenar eventos na ausência de um relógio físico global[49494949].

- **Relógios Lógicos de Lamport:** Atribuem **valores numéricos (carimbos de tempo)** aos eventos para estabelecer uma **ordem sequencial (parcial)**[50505050].
- **Relógios Vetoriais:** Atribuem um **vetor de tempo a cada nó**, permitindo a identificação de **relações causais** e de dependência entre eventos[515151515151515151].
- **Total Order Broadcast (Difusão de Ordem Total):** Mecanismo que garante que **todas as mensagens sejam entregues na mesma ordem** para todos os destinatários[52]. É fundamental para a

integridade de sistemas distribuídos (ex.: transações financeiras)[53535353].

- **Algoritmos de Consenso (Paxos e Raft):** Garantem que todos os nós do sistema **concordem sobre um estado compartilhado** (consistência), mesmo diante de falhas[54545454545454545454]. O

Raft foi projetado para ser mais **simples e claro** que o Paxos[55].

3. Eleição de Coordenadores

O processo de eleição escolhe um

processo líder para gerenciar tarefas e tomar decisões críticas, essencial para a continuidade em caso de falha do líder[565656565656565656].

- **Algoritmo de Bully:** Prioriza o processo com o **maior ID** (ou maior capacidade) como líder[57]. É eficaz em redes menores para uma

rápida substituição do coordenador[58585858].

- **Algoritmo em Anel:** Organiza os processos em uma **estrutura cíclica** (anel) para a eleição[59]. Reduz a complexidade da comunicação, sendo usado em redes estruturadas[60606060].

- **Detecção de Falhas:** É crucial para iniciar a eleição[61]. O mecanismo de

heartbeat (batimento cardíaco) é amplamente utilizado, onde processos enviam sinais periódicos para monitorar a conectividade[62626262].

4. Deadlock em Sistemas Distribuídos

- **Deadlock:** Ocorre quando **múltiplos processos ficam bloqueados**, aguardando indefinidamente por recursos que estão sendo utilizados uns pelos outros, criando um **ciclo de espera**[63636363].

- **Mecanismos de Prevenção:** Buscam evitar que o problema ocorra[64]. Estratégias como a

alocação ordenada de recursos e a liberação preventiva de recursos mitigam ciclos de dependência mútua[65656565].

- **Mecanismos de Detecção e Resolução:**

    - **Algoritmo de Chandy-Misra-Haas:** Algoritmo distribuído que utiliza a troca de mensagens entre processos para identificar **ciclos de espera**[66666666].

    - **Preempção:** Libera recursos de um processo em execução para atender a outro de **maior prioridade**[67676767].

    - **Gráficos de Alocação de Recursos:** Mapeiam as relações entre processos e recursos para identificar visualmente os ciclos de dependência[68].

    - A combinação com

checkpointing facilita a recuperação, restaurando o sistema a um estado seguro[69].

Unidade 3: Tolerância a Falhas e Recuperação

Esta unidade trata da capacidade de um sistema distribuído de continuar operando normalmente, mesmo na presença de falhas, garantindo a

confiabilidade e a **disponibilidade**[70707070707070707070707070707070].

1. Tipos de Falhas em Sistemas Distribuídos

As falhas são classificadas em categorias que podem comprometer a continuidade do serviço[71].

- **Falhas de Rede:** Ocorrem quando a comunicação entre os nós é interrompida ou degradada (ex.: falhas em roteadores, perda de pacotes)[72727272].
- **Falhas de Processo:** Comportamento anômalo de um processo (ex.: **falha por *crash***, onde o processo para abruptamente, ou **falhas lógicas** por erros no código)[73737373].
- **Falhas de Hardware:** O equipamento físico falha (ex.: servidores, discos rígidos)[74].
- **Efeitos das Falhas:**
    - **Interrupção de Serviços:** Afeta a capacidade de fornecer um serviço contínuo[75].
    - **Perda de Dados e Inconsistência:** Especialmente problemática em sistemas com replicação de dados[76].
    - **Sobrecarga de Componentes Saudáveis:** Nós remanescentes precisam assumir a carga do nó falho, resultando em degradação de desempenho[77777777].

2. Técnicas de Tolerância a Falhas

- **Replicação de Dados e Serviços:** Cria e mantém **cópias redundantes** de dados ou instâncias de serviços em diferentes nós[78].
    - **Síncrona:** Todas as réplicas são atualizadas **simultaneamente**[79].
    - **Assíncrona:** Réplicas são atualizadas em **momentos diferentes**[80].
- **Verificação de Consistência:** Assegura que todas as cópias de dados replicados permaneçam consistentes[81].
    - **Consistência Forte:** Garante que todas as réplicas reflitam o mesmo estado **imediatamente** após a escrita[82].
    - **Consistência Eventual:** Réplicas convergem para o mesmo estado após um período de tempo[83].
    - **Algoritmos de Coordenação (Paxos e Raft):** Garantem que as réplicas concordem sobre o estado atual[84].
- **Checkpointing e Rollback Recovery:**
    - **Checkpointing:** Salvamento **periódico** do estado do sistema, criando pontos de recuperação[85]. Pode ser

coordenado (sincronizado entre nós) ou **independente** (autônomo por nó)[86868686].

    - **Rollback Recovery:** Restaura o sistema ao último **checkpoint consistente** após uma falha[87].

3. Sistemas Tolerantes a Falhas

- **Algoritmos de Replicação (Avançados):**
    - **Primary-Backup:** Uma réplica **principal (*primary*)** coordena e replica as operações para réplicas **secundárias (*backups*)**. Usado em sistemas que exigem consistência forte (ex.: MySQL Cluster)[88888888].
    - **Replicação Ativa:** Todas as réplicas processam as requisições **simultaneamente** e de forma independente[89]. Usado em sistemas de mensagens (ex.: Apache Kafka, RabbitMQ)[90].
- **Tolerância a Falhas Bizantinas (BFT):** Lida com o cenário mais desafiador, incluindo **comportamentos arbitrários ou maliciosos** dos componentes[91].
    - **PBFT (Practical Byzantine Fault Tolerance):** Algoritmo BFT que garante consenso, operando em fases como **Pré-preparação, Preparação e Confirmação**[929292929292929292]. É a base para soluções em redes

blockchain (ex.: Hyperledger Fabric)[93].

- **Acordo e Consenso:** Algoritmos como **Paxos** e **Raft** asseguram que todas as réplicas concordem em um único valor[94949494].

4. Detecção e Recuperação de Falhas

- **Detecção de Falhas:** É crucial para iniciar a recuperação[95959595].
    - **Algoritmos de verificação de pulsos (Heartbeat):** Processos emitem sinais periodicamente; a falta de pulso indica falha[96].
    - **Detecção síncrona:** Baseada em limites de tempo (*timeout*) predefinidos para operações[97979797].
- **Recuperação Automática e Reinicialização:**
    - **Recuperação proativa:** Tenta evitar falhas (ex.: reinicialização periódica de componentes)[98].
    - **Recuperação reativa:** Responde à falha após sua ocorrência (ex.: *rollback* ou reinicialização)[99].
    - **Ferramentas: Kubernetes** (suporte nativo, reinicia *pods* falhos) [100100100100]e

Zookeeper (usa *heartbeat* para monitoramento)[101].

- **Reconfiguração de Sistemas:** Processo essencial para adaptar a configuração do sistema após falhas, redirecionando recursos ou ativando nós de backup[102].
    - **Kubernetes (kubectl drain):** Usado para *drenar* (remover a carga) um nó e redistribuir as tarefas para outros nós operacionais[103].
    - **Automação (*Terraform* e *Ansible*):** Usadas para gerenciar a infraestrutura como código e automatizar a reconfiguração[104104104104].

Unidade 4: Sistemas Distribuídos Modernos

Esta unidade explora as tendências e arquiteturas que definem os sistemas distribuídos atuais, com foco em Big Data, Nuvem, Blockchains e Microsserviços[105].

1. Armazenamento Distribuído e Big Data

- **Armazenamento Distribuído:** Divide dados em fragmentos armazenados em múltiplos servidores para garantir **redundância, resiliência e escalabilidade**[106].
    - **HDFS (Hadoop Distributed File System):** Sistema de arquivos para Big Data, seguindo o modelo mestre-escravo, com **NameNode** (metadados) e **DataNodes** (dados reais)[107107107107].
    - **Ceph:** Armazenamento unificado (blocos, arquivos, objetos), descentralizado, que utiliza o algoritmo **CRUSH**[108108108108].
- **Big Data e os 5 Vs:** Volume massivo de dados que é processado e analisado[109].
    - **Volume:** Imenso volume de dados gerados[110].
    - **Velocidade:** Rapidez com que os dados são gerados e processados[111].
    - **Variedade:** Diversos formatos de dados (estruturados e não estruturados)[112].
    - **Veracidade:** Precisão e confiabilidade dos dados[113].
    - **Valor:** Potencial de gerar *insights* e decisões estratégicas[114].
- **MapReduce:** Modelo de programação e *framework* para processar grandes conjuntos de dados de forma **distribuída e paralela**[115]. Funciona em duas etapas:

Map (mapeamento) e **Reduce** (redução)[116].

- **Particionamento e Sharding:**
    - **Particionamento Horizontal (Sharding):** Divide as **linhas** de uma tabela em fragmentos menores (*shards*), distribuídos entre vários servidores[117117117117].
    - **Particionamento Vertical:** Divide uma tabela em subconjuntos de **colunas**[118].

2. Computação em Nuvem e Virtualização

- **Computação em Nuvem (*Cloud Computing*):** Acesso a recursos de TI **sob demanda** via internet[119].
    - **Modelos de Serviço:**
        - **SaaS (Software as a Service):** Software pronto para uso (ex.: e-mail)[120].
        - **PaaS (Platform as a Service):** Ambiente para desenvolvimento e *deploy*[121].
        - **IaaS (Infrastructure as a Service):** Acesso a infraestrutura básica (servidores virtuais)[122].
- **Virtualização:** Permite a emulação de recursos físicos (servidores, hardware) em múltiplas instâncias isoladas[123]. O

Hypervisor é o *software* que gerencia a criação e alocação de recursos em **Máquinas Virtuais (VMs)**[124].

- **Conteinerização:** Tecnologia de virtualização **leve** que empacota o aplicativo com todas as suas dependências[125].
    - **Docker:** Plataforma popular para criação e execução de contêineres[126].
    - **Kubernetes (K8s):** Plataforma de **orquestração** de contêineres[127]. Gerencia a implantação, escalabilidade e operação de aplicações em

clusters[128].

3. Blockchain e Criptomoedas

- **Blockchain:** Livro-razão digital **distribuído** e **imutável** que registra transações de forma transparente e segura[129].
    - **Características: Descentralização**, **transparência** e **segurança** (por meio de criptografia **hash**)[130130130130].
- **Contratos Inteligentes:** Programas autoexecutáveis em redes *blockchain* que executam ações predeterminadas quando condições específicas são atendidas[131].
- **Consenso em Blockchain:** Processo para que os participantes cheguem a um acordo sobre o estado do registro, validando transações[132].
    - **Proof of Work (PoW):** Mineradores resolvem problemas matemáticos complexos (*hashing*) para validar transações. Consome alto poder computacional e energia (ex.: Bitcoin)[133].
    - **Proof of Stake (PoS):** Validadores são escolhidos com base na quantidade de criptomoedas que eles bloqueiam (*staking*). É mais eficiente em energia e mais escalável (ex.: Ethereum 2.0)[134134134134].

4. Microsserviços e Arquiteturas Distribuídas Modernas

- **Microsserviços:** Estilo arquitetônico que divide uma aplicação em **pequenos serviços independentes**[135].
    - **Vantagens: Escalabilidade seletiva**, **resiliência** (falha em um não derruba o todo) e **independência tecnológica**[136136136136].
- **Comunicação entre Microsserviços (APIs):**
    - **REST (Representational State Transfer):** Estilo arquitetural baseado em **HTTP** (usando métodos GET, POST, PUT, DELETE) e formato **JSON/XML**[137137137137137137137137137].
    - **gRPC (gRPC Remote Procedure Call):** Framework de alta performance baseado em **HTTP/2** que usa comunicação **binária** (*Protocol Buffers*)[138138138138].
    - **Message Brokers (*Kafka*, *RabbitMQ*):** Sistemas que permitem a comunicação **assíncrona** entre serviços, desacoplando os processos (modelo produtor-consumidor)[139139139139].

- **Orquestração de Contêineres:**
    - **Kubernetes (K8s):** Plataforma de orquestração avançada, que automatiza o *deploy*, escalabilidade e gerenciamento de contêineres[140140140140].
    - **Docker Swarm:** Ferramenta de orquestração mais simples e integrada ao Docker, usando o algoritmo de consenso **Raft**[141141141141141141141141141].